

EXPRESS ANALYTICS

AI/ML Engineer

TAKE-HOME ASSIGNMENT

AI Travel Planner

Build a multi-agent travel planning system with human-in-the-loop approval

Role AI/ML Engineer

Duration 1.5/2 days

Submission GitHub Repository +
README

Stack Python, LangGraph, FastAPI

Please read the entire document before starting.

Overview

Build an AI-powered travel planning assistant that takes a user's travel preferences and produces a complete trip plan. The system must use a LangGraph workflow with an orchestrator, two specialized agents, and human-in-the-loop (HITL) approval before finalizing the plan. The entire system should be served via a FastAPI application.

This assignment evaluates your ability to design multi-agent systems, implement stateful workflows with interrupts, and deliver production-quality API services. There is no single correct architecture — we want to see how you think.

Scenario

A user submits a travel request with the following information: destination, travel dates, budget range, interests/preferences, and number of travelers. Your system should research the destination, build a day-by-day itinerary, and present it to the user for approval before producing the final plan.

System Architecture

Orchestrator

The orchestrator manages the overall workflow using LangGraph's StateGraph. It should:

- Receive and validate the incoming travel request
- Route work to the appropriate agent based on the current workflow stage
- Manage state transitions between research, planning, and approval phases
- Handle the HITL interrupt — pause execution, surface the draft plan for review, and resume based on user feedback (approve, reject with comments, or modify)
- Produce the final output after approval

Agent 1: Research Agent

This agent gathers destination intelligence. It has access to two tools:

Tool 1: Web Search (Mandatory)

Integrate either **Serper API** (serper.dev) or **Exa API** (exa.ai) for real-time web search. The agent should use this to research the destination — attractions, local tips, safety information, weather, seasonal considerations, and anything else relevant to the trip.

Tool 2: Your Choice

Implement a second tool that complements the research capability. This could be a weather API, a currency converter, a flight/hotel price lookup, a mock knowledge base, or anything else you think adds value. The design of this tool is entirely up to you.

Agent 2: Itinerary Planner Agent

This agent takes the research output and constructs a structured trip plan. It has access to two tools:

Tool 1: Your Choice

Design a tool that helps with itinerary construction — perhaps a places/activities database, a distance/travel-time calculator, a budget allocator, or a scheduling optimizer.

Tool 2: Your Choice

Design a second tool that adds depth to the planning — perhaps a restaurant recommender, a local events lookup, a packing list generator, or anything else you think improves the output quality.

Human-in-the-Loop (HITL)

After the Itinerary Planner Agent produces a draft plan, the workflow must pause and present the plan to the user for review. The HITL step should support:

1. Approve — the plan is accepted as-is and the workflow proceeds to final output
2. Reject with feedback — the user provides comments, and the workflow routes back to the appropriate agent(s) for revision
3. Modify — the user can adjust specific parts of the plan (e.g., swap a day's activity, change a hotel preference)

Think carefully about how HITL interacts with your API design. The workflow must persist state across the pause — this is a core evaluation criterion.

API Requirements

Serve the system via FastAPI. At minimum, your API should support the following interactions:

Method	Endpoint	Purpose	Notes
POST	/plan	Submit a new travel request	Returns a session/plan ID
GET	/plan/{id}	Get current plan status and draft	Should reflect workflow stage
POST	/plan/{id}/review	Submit HITL feedback	Approve, reject, or modify
GET	/plan/{id}/final	Retrieve the finalized plan	Only available after approval

You may add additional endpoints as you see fit. Consider error handling, input validation, and meaningful status codes.

Deliverables

4. A GitHub repository (public or private — invite us if private) containing all source code
5. A README.md with: project overview, architecture diagram or description, setup instructions (environment, API keys, dependencies), how to run the application, example API requests and responses, and a section on design decisions and tradeoffs
6. A working FastAPI application that can be run locally
7. A brief write-up (can be in the README) or a video recording explaining: what you would improve with more time, how you would handle production concerns and any assumptions you made.

Reference: APIs You Can Use

Below are some APIs that may be useful for building your tools. You are not limited to these — use whatever you think is appropriate.

Web Search (Mandatory, or choose something better)

API / Tool	Purpose	Free Tier / Notes
Serper (serper.dev)	Google Search results via API	2,500 free queries on signup
Exa (exa.ai)	Neural search with content retrieval	1,000 free searches/month

Important Notes

- You may use any LLM provider for the agents (OpenAI, Anthropic, Groq, etc.)
- Focus on architecture and workflow design over UI polish
- If you run into time constraints, prioritize the orchestrator + HITL flow over adding more tools
- We value clear thinking and honest documentation of tradeoffs over feature completeness

Good luck! We look forward to reviewing your work.